

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA KỸ THUẬT ĐIỆN TỬ 1



BÁO CÁO CUỐI KỲ

HỌC PHẦN: CẢM BIẾN VÀ CƠ CẤU CHẤP HÀNH

**NGHIÊN CỨU VÀ XÂY DỰNG HỆ THỐNG
ROBOT TỰ HÀNH TURTLEBOT4 TRONG MÔI
TRƯỜNG MÔ PHỎNG SỬ DỤNG ROS2, SLAM,
NAV2 VÀ YOLOV8**

Giảng viên hướng dẫn: TS. Phạm Hoàng Anh

Sinh viên thực hiện: Nguyễn Hữu Hoàng Anh

Mã sinh viên: B23DCDK007

Lớp: D23CQDKRB

Hà Nội, ngày 04 tháng 06 năm 2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn chân thành đến Thầy **TS. Phạm Hoàng Anh**, giảng viên hướng dẫn học phần *Cảm biến và Cơ cấu chấp hành*, đã tận tình giảng dạy, định hướng và truyền đạt các kiến thức nền tảng về cảm biến, cơ cấu chấp hành, hệ thống đo lường, điều khiển chuyển động và ứng dụng của chúng trong các hệ thống robot hiện đại.

Học phần đã giúp em hiểu rõ hơn vai trò của cảm biến trong việc thu nhận thông tin từ môi trường, vai trò của cơ cấu chấp hành trong biến đổi tín hiệu điều khiển thành chuyển động vật lý, cũng như mối liên hệ chặt chẽ giữa nhận thức, định vị, lập kế hoạch và điều khiển trong một hệ robot tự hành. Những kiến thức này là cơ sở quan trọng để em triển khai đề tài nghiên cứu hệ thống robot tự hành TurtleBot4 trong môi trường mô phỏng sử dụng ROS2, SLAM, Nav2 và YOLOv8.

Em cũng xin cảm ơn Khoa Kỹ thuật Điện tử 1, Học viện Công nghệ Bưu chính Viễn thông đã tạo điều kiện học tập, nghiên cứu và thực hành trong quá trình thực hiện báo cáo. Mặc dù đã cố gắng hoàn thiện báo cáo một cách nghiêm túc, bài làm vẫn khó tránh khỏi thiếu sót do giới hạn về thời gian và kinh nghiệm triển khai thực nghiệm. Em kính mong nhận được góp ý của thầy để tiếp tục hoàn thiện kiến thức và năng lực nghiên cứu trong lĩnh vực robotics.

Hà Nội, ngày 04 tháng 06 năm 2026

Sinh viên thực hiện

Nguyễn Hữu Hoàng Anh

LỜI MỞ ĐẦU

Robot tự hành đang trở thành một hướng phát triển quan trọng của kỹ thuật robot hiện đại, đặc biệt trong các bài toán kho vận thông minh, giám sát, tuần tra, dịch vụ và hỗ trợ con người. Một robot tự hành không chỉ cần cơ cấu chuyển động ổn định mà còn cần hệ cảm biến đủ tin cậy để nhận biết môi trường, định vị bản thân, phát hiện đối tượng và ra quyết định hành động. Vì vậy, việc nghiên cứu robot tự hành là một ví dụ điển hình cho sự kết hợp giữa cảm biến, cơ cấu chấp hành và thuật toán điều khiển.

Báo cáo này trình bày quá trình xây dựng hệ thống robot tự hành TurtleBot4 trong môi trường mô phỏng. Hệ thống sử dụng ROS2 làm nền tảng tích hợp phần mềm, Gazebo/Ignition làm môi trường mô phỏng, LiDAR và camera RGB-D làm nguồn dữ liệu cảm biến, SLAM Toolbox để xây dựng bản đồ, Nav2 để điều hướng, YOLOv8 để nhận diện mục tiêu và một Mission Manager để điều phối hành vi tuần tra, phát hiện và tiếp cận mục tiêu.

Dưới góc nhìn của học phần *Cảm biến và Cơ cấu chấp hành*, trọng tâm báo cáo không chỉ là phần mềm robot mà còn là phân tích mối quan hệ hệ thống: cảm biến cung cấp dữ liệu đo, thuật toán xử lý biến dữ liệu đo thành thông tin trạng thái, bộ lập kế hoạch tạo lệnh chuyển động, và cơ cấu chấp hành của robot thực hiện lệnh để tạo chuyển động trong môi trường. Cách tiếp cận này giúp đánh giá hệ robot như một hệ cơ điện tử hoàn chỉnh thay vì chỉ là một tập hợp các module rời rạc.

Nội dung báo cáo gồm tám chương: tổng quan đề tài; cơ sở lý thuyết; thiết kế kiến trúc hệ thống; hệ thống cảm biến; cơ cấu chấp hành và điều khiển chuyển động; triển khai nhiệm vụ tự hành; kết quả mô phỏng và đánh giá; kết luận và hướng phát triển.

MỤC LỤC

LỜI CẢM ƠN	i
LỜI MỞ ĐẦU	ii
1 TỔNG QUAN ĐỀ TÀI	1
1.1 Bối cảnh nghiên cứu	1
1.2 Lý do chọn đề tài	1
1.3 Mục tiêu của đề tài	1
1.4 Phạm vi nghiên cứu	2
1.5 Đóng góp chính của báo cáo	2
2 CƠ SỞ LÝ THUYẾT	3
2.1 Robot tự hành và vòng kín cảm biến–chấp hành	3
2.2 Cảm biến trong robot di động	3
2.2.1 LiDAR	3
2.2.2 Camera RGB-D	3
2.2.3 Odometry và encoder	4
2.3 Hệ tọa độ và TF trong ROS2	4
2.4 SLAM	4
2.5 Nav2 và điều hướng robot	5
2.6 YOLOv8 trong nhận diện đối tượng	5
2.7 Cơ cấu chấp hành robot vi sai	5
2.8 Kết luận chương	6
3 THIẾT KẾ KIẾN TRÚC HỆ THỐNG	6
3.1 Tổng quan kiến trúc	6
3.2 Luồng dữ liệu chính	7
3.3 Sơ đồ khối hệ thống	7
3.4 Môi trường mô phỏng	7

3.5	Cấu hình launch tổng hợp	8
3.6	Kết luận chương	8
4	HỆ THỐNG CẢM BIẾN VÀ XỬ LÝ DỮ LIỆU	8
4.1	Vai trò của cảm biến trong đề tài	8
4.2	LiDAR và dữ liệu /scan	8
4.3	Camera RGB-D OAK-D	9
4.4	Nhận diện đối tượng bằng YOLOv8n	9
4.5	Định vị mục tiêu từ RGB-D	9
4.6	TF và hợp nhất thông tin cảm biến	10
4.7	Đánh giá kỹ thuật hệ cảm biến	10
4.8	Kết luận chương	10
5	CƠ CẤU CHẤP HÀNH VÀ ĐIỀU KHIỂN CHUYỂN ĐỘNG	10
5.1	Cấu trúc chấp hành của TurtleBot4	10
5.2	Mô hình động học robot vi sai	11
5.3	Nav2 như lớp điều khiển chuyển động cấp cao	11
5.4	Patrol Node	12
5.5	Mission Manager và điều khiển hành vi	12
5.6	Quan hệ giữa cảm biến và chấp hành	12
5.7	Kết luận chương	13
6	TRIỂN KHAI NHIỆM VỤ TỰ HÀNH	13
6.1	Mục tiêu nhiệm vụ	13
6.2	Quy trình khởi động hệ thống	13
6.3	SLAM và bản đồ môi trường	14
6.4	Navigation và tuần tra	14
6.5	Nhận diện và định vị mục tiêu	15
6.6	Mission Manager	15
6.7	Các topic kiểm tra quan trọng	15

6.8	Kết luận chương	16
7	KẾT QUẢ MÔ PHỎNG VÀ ĐÁNH GIÁ	16
7.1	Kết quả đạt được	16
7.2	Đánh giá hệ cảm biến	16
7.3	Đánh giá cơ cấu chấp hành và điều khiển	17
7.4	Đánh giá kiến trúc phần mềm	17
7.5	Hạn chế	17
7.6	Hướng phát triển	18
7.7	Kết luận chương	18
8	KẾT LUẬN	18
8.1	Kết luận chung	18
8.2	Các nội dung đã hoàn thành	19
8.3	Ý nghĩa đối với học phần	19
8.4	Hạn chế và định hướng tiếp theo	20
8.5	Lời kết	20
	PHỤ LỤC A: CẤU TRÚC SOURCE CODE	21
	PHỤ LỤC B: CÁC LỆNH CHẠY CHÍNH	22
	PHỤ LỤC C: MỘT SỐ LỖI VÀ CÁCH KHẮC PHỤC	24
	TÀI LIỆU THAM KHẢO	25

1. TỔNG QUAN ĐỀ TÀI

1.1 Bối cảnh nghiên cứu

Trong những năm gần đây, robot tự hành đã được ứng dụng rộng rãi trong các lĩnh vực như kho vận, sản xuất thông minh, giám sát an ninh, chăm sóc y tế, dịch vụ trong nhà và nghiên cứu giáo dục. Điểm khác biệt cơ bản giữa robot tự hành và các cơ cấu máy tự động truyền thống nằm ở khả năng nhận thức môi trường, tự định vị, lập kế hoạch đường đi và điều khiển chuyển động một cách thích nghi.

Một robot di động hiện đại thường bao gồm ba lớp chức năng chính. Lớp thứ nhất là lớp cảm biến, có nhiệm vụ thu nhận dữ liệu từ môi trường và từ trạng thái nội tại của robot. Lớp thứ hai là lớp xử lý và ra quyết định, bao gồm định vị, lập bản đồ, nhận diện vật thể, lập kế hoạch và quản lý nhiệm vụ. Lớp thứ ba là lớp chấp hành, chuyển đổi lệnh điều khiển thành chuyển động thực tế thông qua động cơ, bánh xe và hệ truyền động.

Trong học phần *Cảm biến và Cơ cấu chấp hành*, việc nghiên cứu một hệ robot tự hành có ý nghĩa thực tiễn cao vì hệ thống này thể hiện đầy đủ vòng kín cảm biến–xử lý–điều khiển–chấp hành. Dữ liệu từ LiDAR, camera, odometry và TF không chỉ được quan sát độc lập mà còn được kết hợp để phục vụ bài toán SLAM, navigation và tiếp cận mục tiêu.

1.2 Lý do chọn đề tài

Đề tài lựa chọn TurtleBot4 vì đây là nền tảng robot di động phổ biến trong nghiên cứu và đào tạo robotics. TurtleBot4 có cấu trúc cơ khí dạng robot vi sai, tích hợp các cảm biến quan trọng như LiDAR, camera RGB-D, encoder/odometry và hỗ trợ tốt hệ sinh thái ROS2. Điều này cho phép người học khảo sát tương đối đầy đủ mối quan hệ giữa cảm biến và cơ cấu chấp hành trong một hệ robot di động.

Việc triển khai trong môi trường mô phỏng Gazebo/Ignition giúp giảm chi phí phần cứng, giảm rủi ro khi thử nghiệm, đồng thời cho phép kiểm tra các thuật toán trước khi triển khai trên robot thật. Bên cạnh đó, ROS2 Humble, SLAM Toolbox, Nav2 và YOLOv8 là các công cụ hiện đại, có tính ứng dụng cao trong thực tế công nghiệp và nghiên cứu.

1.3 Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài là xây dựng và phân tích một hệ thống robot tự hành TurtleBot4 có khả năng tuần tra, nhận diện và tiếp cận mục tiêu trong môi trường mô

phòng. Các mục tiêu cụ thể gồm:

- Thiết lập môi trường mô phỏng TurtleBot4 Lite trong Gazebo/Ignition.
- Tích hợp các nguồn dữ liệu cảm biến: LiDAR, camera RGB-D, odometry và TF.
- Sử dụng SLAM Toolbox để xây dựng bản đồ môi trường.
- Sử dụng Nav2 để điều hướng robot trên bản đồ đã có.
- Tích hợp YOLOv8n để nhận diện đối tượng mục tiêu từ ảnh RGB.
- Ước lượng vị trí mục tiêu trong không gian 3D dựa trên ảnh depth và thông số nội tại camera.
- Xây dựng Mission Manager để điều phối hành vi tuần tra, tạm dừng tuần tra, tiếp cận mục tiêu và quay lại nhiệm vụ ban đầu.

1.4 Phạm vi nghiên cứu

Đề tài tập trung vào mô phỏng và tích hợp hệ thống. Robot sử dụng là TurtleBot4 Lite trong môi trường warehouse của Gazebo/Ignition. Phần mềm triển khai trên ROS2 Humble. Nhận diện đối tượng dùng YOLOv8n pretrained trên COCO dataset, trong đó lớp *person* được ưu tiên như mục tiêu chính. Hệ thống chưa triển khai trên phần cứng TurtleBot4 thật, chưa tối ưu sâu tham số Nav2 và chưa xử lý tracking nhiều mục tiêu động.

1.5 Đóng góp chính của báo cáo

Báo cáo không chỉ mô tả cách chạy một hệ thống ROS2 mà còn phân tích vai trò kỹ thuật của từng thành phần dưới góc nhìn cảm biến và cơ cấu chấp hành. Các đóng góp chính gồm:

- Đề xuất kiến trúc module cho hệ robot tự hành TurtleBot4.
- Phân tích chuỗi xử lý cảm biến từ LiDAR/camera đến bản đồ, pose và marker mục tiêu.
- Phân tích cơ cấu chấp hành vi sai và cách Nav2 tạo lệnh vận tốc cho robot.
- Xây dựng luồng nhiệm vụ tự hành kết hợp patrol, perception, localization và navigation.
- Đánh giá các kết quả đạt được, hạn chế và hướng phát triển phù hợp cho robot thật.

2. CƠ SỞ LÝ THUYẾT

2.1 Robot tự hành và vòng kín cảm biến–chấp hành

Robot tự hành là hệ cơ điện tử có khả năng di chuyển trong môi trường mà không cần điều khiển trực tiếp liên tục từ con người. Để đạt được điều này, robot phải thực hiện chu trình nhận thức và hành động gồm: đo lường môi trường, ước lượng trạng thái, lập bản đồ, lập kế hoạch, phát lệnh điều khiển và thực thi chuyển động.

Dưới góc nhìn hệ thống, robot tự hành có thể được biểu diễn bởi vòng kín:

Môi trường → Cảm biến → Xử lý/Ước lượng → Điều khiển → Cơ cấu chấp hành → Môi trường

Trong vòng kín này, cảm biến quyết định chất lượng thông tin đầu vào, còn cơ cấu chấp hành quyết định khả năng biến quyết định điều khiển thành chuyển động thực tế. Sai số hoặc độ trễ ở một khâu đều có thể ảnh hưởng trực tiếp đến tính ổn định và độ tin cậy của toàn bộ robot.

2.2 Cảm biến trong robot di động

2.2.1 LiDAR

LiDAR đo khoảng cách bằng cách phát tia laser và tính toán khoảng cách tới vật cản dựa trên thời gian bay hoặc nguyên lý tam giác lượng. Trong robot di động, LiDAR 2D thường tạo ra bản tin LaserScan, biểu diễn tập hợp khoảng cách theo các góc quét trong mặt phẳng ngang.

Dữ liệu LiDAR có vai trò quan trọng trong SLAM, tránh vật cản và xây dựng costmap. Với TurtleBot4, topic /scan cung cấp thông tin khoảng cách xung quanh robot. Các tham số như tầm đo tối thiểu, tầm đo tối đa, độ phân giải góc và nhiễu đo ảnh hưởng trực tiếp đến chất lượng bản đồ và độ ổn định của navigation.

2.2.2 Camera RGB-D

Camera RGB-D cung cấp đồng thời ảnh màu và ảnh chiều sâu. Ảnh RGB hỗ trợ nhận diện đối tượng bằng các mô hình học sâu như YOLOv8, còn ảnh depth cho phép ước lượng khoảng cách từ camera tới đối tượng. Với mô hình camera lỗ kim, một điểm ảnh (u, v) có độ sâu Z được chuyển sang tọa độ 3D trong hệ camera theo công thức:

$$X = \frac{(u - c_x)Z}{f_x}, \quad Y = \frac{(v - c_y)Z}{f_y}, \quad Z = Z$$

Trong đó f_x, f_y là tiêu cự theo pixel, c_x, c_y là tọa độ tâm ảnh. Các tham số này được lấy từ bản tin CameraInfo. Trong đề tài, công thức này được dùng để định vị mục tiêu 3D từ bbox YOLO và ảnh depth.

2.2.3 Odometry và encoder

Odometry là ước lượng chuyển động tương đối của robot theo thời gian, thường dựa trên encoder bánh xe và mô hình động học. Với robot vi sai, vận tốc tuyến tính v và vận tốc góc ω được liên hệ với vận tốc bánh trái v_l , bánh phải v_r bởi:

$$v = \frac{v_r + v_l}{2}, \quad \omega = \frac{v_r - v_l}{L}$$

Trong đó L là khoảng cách giữa hai bánh chủ động. Odometry có ưu điểm là tần số cao và đáp ứng nhanh, nhưng sai số tích lũy theo thời gian do trượt bánh, sai số encoder và mô hình hóa chưa hoàn hảo. Vì vậy, odometry thường được kết hợp với LiDAR/camera trong SLAM hoặc localization.

2.3 Hệ tọa độ và TF trong ROS2

Trong robot, dữ liệu từ nhiều cảm biến chỉ có ý nghĩa khi được quy về các hệ tọa độ nhất quán. ROS2 sử dụng TF/TF2 để quản lý quan hệ hình học giữa các frame như map, odom, base_link, oakd_link, rplidar_link.

Quan hệ điển hình trong robot di động là:

$$\text{map} \rightarrow \text{odom} \rightarrow \text{base_link} \rightarrow \text{sensor_frame}$$

Frame map là hệ toàn cục ổn định dùng cho navigation. Frame odom liên tục nhưng có thể trôi. Frame base_link gắn với thân robot. Các frame cảm biến gắn với vị trí lắp đặt của LiDAR và camera. Việc biến đổi chính xác giữa các frame là điều kiện bắt buộc để robot có thể dùng dữ liệu cảm biến cho lập bản đồ, hiển thị RViz và điều hướng.

2.4 SLAM

SLAM, viết tắt của *Simultaneous Localization and Mapping*, là bài toán robot vừa xây dựng bản đồ môi trường vừa ước lượng vị trí của chính nó trong bản đồ đó. Với

LiDAR 2D, SLAM thường dựa trên scan matching giữa các lần quét liên tiếp, kết hợp với odometry để giảm không gian tìm kiếm.

Trong đề tài, SLAM Toolbox được sử dụng với các thành phần chính: scan matching, quản lý pose graph, phát hiện loop closure và tối ưu hóa đồ thị. Kết quả của SLAM là occupancy grid map và transform map $\rightarrow$ odom. Bản đồ này sau đó được dùng cho Nav2 để lập kế hoạch đường đi.

2.5 Nav2 và điều hướng robot

Nav2 là framework điều hướng trong ROS2, cung cấp các chức năng localization, global planning, local control, recovery behavior và action interface. Trong hệ thống này, Mission Manager và Patrol Node gửi goal đến Nav2 thông qua action NavigateToPose. Nav2 sau đó tính đường đi trên bản đồ, tạo lệnh vận tốc `/cmd_vel` và điều khiển robot di chuyển tới mục tiêu.

Về bản chất, Nav2 là lớp trung gian giữa thuật toán lập kế hoạch và cơ cấu chấp hành. Nó chuyển yêu cầu vị trí đích thành chuỗi lệnh vận tốc phù hợp với giới hạn động học và môi trường xung quanh robot.

2.6 YOLOv8 trong nhận diện đối tượng

YOLOv8 là mô hình phát hiện đối tượng một giai đoạn, có khả năng dự đoán bounding box, nhãn lớp và độ tin cậy trực tiếp từ ảnh đầu vào. Trong đề tài, YOLOv8n được lựa chọn vì kích thước nhỏ, tốc độ nhanh và phù hợp với bài toán mô phỏng thời gian thực.

Kết quả nhận diện được đóng gói dưới dạng `Detection2DArray`. Mỗi detection gồm tọa độ tâm bbox, kích thước bbox, class id và confidence. Khi kết hợp với ảnh depth, bbox 2D được nâng thành vị trí 3D tương đối so với camera.

2.7 Cơ cấu chấp hành robot vi sai

TurtleBot4 sử dụng cấu trúc truyền động vi sai gồm hai bánh chủ động độc lập. Cơ cấu này có ưu điểm đơn giản, dễ điều khiển, bán kính quay nhỏ và phù hợp với môi trường trong nhà. Lệnh điều khiển thường ở dạng vận tốc tuyến tính v và vận tốc góc ω . Bộ điều khiển cấp thấp chuyển đổi (v, ω) thành vận tốc góc từng bánh.

Nếu bán kính bánh xe là r , khoảng cách hai bánh là L , vận tốc góc bánh trái/phải lần lượt là ω_l, ω_r , ta có:

$$\omega_r = \frac{v + \frac{L}{2}\omega}{r}, \quad \omega_l = \frac{v - \frac{L}{2}\omega}{r}$$

Mô hình này là nền tảng để hiểu cách lệnh `/cmd_vel` từ Nav2 được chuyển thành chuyển động vật lý của robot.

2.8 Kết luận chương

Chương này đã trình bày các cơ sở lý thuyết quan trọng liên quan đến cảm biến, cơ cấu chấp hành và robot tự hành. Các nội dung gồm LiDAR, camera RGB-D, odometry, TF, SLAM, Nav2, YOLOv8 và mô hình truyền động vi sai. Đây là nền tảng để phân tích kiến trúc và quá trình triển khai hệ thống TurtleBot4 trong các chương tiếp theo.

3. THIẾT KẾ KIẾN TRÚC HỆ THỐNG

3.1 Tổng quan kiến trúc

Hệ thống được thiết kế theo kiến trúc module trên ROS2, trong đó mỗi chức năng chính được đóng gói thành một package riêng. Cách tổ chức này giúp hệ thống dễ mở rộng, dễ kiểm thử và phù hợp với tư duy thiết kế robot hiện đại. Các package chính gồm: `tb4_bringup`, `tb4_vision_oak`, `tb4_object_localization`, `tb4_nav_patrol` và `tb4_mission_manager`.

Bảng 3.1 Các package chính của hệ thống

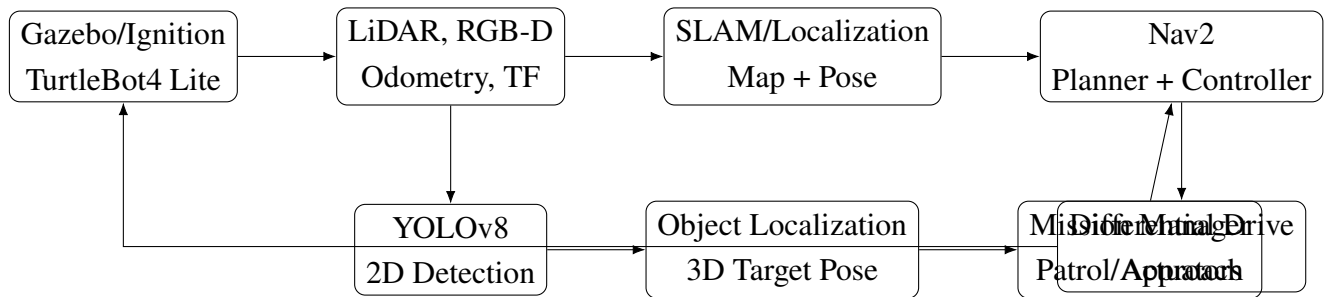
Package	Chức năng
<code>tb4_bringup</code>	Khởi chạy Nav2, localization, RViz, perception, mission manager và cấu hình tổng thể.
<code>tb4_vision_oak</code>	Nhận ảnh RGB từ camera OAK-D, chạy YOLOv8n và publish detection 2D.
<code>tb4_object_localization</code>	Kết hợp detection 2D, ảnh depth và camera intrinsic để ước lượng vị trí mục tiêu 3D.
<code>tb4_nav_patrol</code>	Gửi các waypoint tuần tra cho Nav2 qua action <code>NavigateToPose</code> .
<code>tb4_mission_manager</code>	Điều phối hành vi tuần tra, phát hiện mục tiêu, tiếp cận an toàn và quay lại tuần tra.

3.2 Luồng dữ liệu chính

Luồng dữ liệu trong hệ thống có thể mô tả như sau:

1. Gazebo/Ignition mô phỏng robot, môi trường, LiDAR, camera RGB-D và odometry.
2. LiDAR publish dữ liệu /scan; camera publish ảnh RGB, ảnh depth và /camera_info.
3. SLAM/localization và Nav2 sử dụng bản đồ, TF, odometry và scan để định vị và điều hướng.
4. YOLOv8 nhận ảnh RGB và publish /vision/detected_objects.
5. Object Localization dùng bbox, depth và camera intrinsic để publish /target_object_pose_r và marker RViz.
6. Mission Manager nhận vị trí mục tiêu, tạm dừng Patrol Node và gửi goal tiếp cận an toàn cho Nav2.
7. Nav2 tạo lệnh vận tốc cho robot, hệ chấp hành vi sai thực thi chuyển động.

3.3 Sơ đồ khối hệ thống



Hình 3.1 Kiến trúc tổng quát của hệ thống robot tự hành TurtleBot4

3.4 Môi trường mô phỏng

Môi trường mô phỏng chính là warehouse trong Gazebo/Ignition. Robot được spawn dưới dạng TurtleBot4 Lite với các tham số vị trí ban đầu $x = 2.0$, $y = 0.0$, $z = 0.01$, $yaw = 0.0$. Mô phỏng cho phép đánh giá luồng dữ liệu cảm biến, thuật toán SLAM, Nav2 và mission logic trước khi triển khai trên phần cứng thật.

3.5 Cấu hình launch tổng hợp

File `sim_demo.launch.py` trong package `tb4_bringup` đóng vai trò khởi chạy toàn hệ thống. Launch file này bao gồm localization, Nav2, node phát initial pose, patrol, YOLO detection, object localization, mission manager và RViz nếu được yêu cầu. Việc gom các thành phần vào một launch file giúp giảm lỗi thao tác và đảm bảo thứ tự khởi động hợp lý.

3.6 Kết luận chương

Chương này đã trình bày kiến trúc tổng thể của hệ thống TurtleBot4. Cách tổ chức theo module cho phép tách biệt rõ ràng giữa cảm biến, perception, localization, navigation và mission management. Đây là cơ sở để phân tích chi tiết hệ cảm biến và cơ cấu chấp hành ở các chương tiếp theo.

4. HỆ THỐNG CẢM BIẾN VÀ XỬ LÝ DỮ LIỆU

4.1 Vai trò của cảm biến trong đề tài

Trong hệ thống TurtleBot4, cảm biến không chỉ dùng để quan sát mà còn trực tiếp quyết định khả năng tự hành. LiDAR giúp robot nhận biết vật cản và xây dựng bản đồ; camera RGB-D giúp phát hiện và định vị mục tiêu; odometry cung cấp chuyển động tương đối; TF liên kết dữ liệu giữa các hệ tọa độ. Sự kết hợp này tạo thành nền tảng cho SLAM, Nav2 và Mission Manager.

4.2 LiDAR và dữ liệu `/scan`

LiDAR là cảm biến chủ đạo cho bài toán SLAM và tránh vật cản. Dữ liệu được publish qua topic `/scan`. Trong cấu hình SLAM Toolbox, `scan_topic` được đặt là `/scan`, frame cơ sở là `base_link`, frame bản đồ là `map` và frame odometry là `odom`. Các tham số đáng chú ý gồm độ phân giải bản đồ 0.05 m , tầm đo tối thiểu 0.2 m , tầm đo tối đa 12.0 m , chu kỳ cập nhật bản đồ 2.0 s .

Trong quá trình phát triển, lỗi dữ liệu `/scan` có giá trị bất thường hoặc gần như cố định có thể xuất hiện do cấu hình vị trí LiDAR, collision model hoặc lỗi mô phỏng. Việc kiểm tra topic `/scan`, frame `rplidar_link` và quan sát LaserScan trong RViz là bước quan trọng để đảm bảo chất lượng dữ liệu đầu vào.

4.3 Camera RGB-D OAK-D

Camera OAK-D trong mô phỏng cung cấp ba nhóm dữ liệu chính: ảnh RGB, ảnh depth và thông số nội tại camera. Node `oakd_detection_node` subscribe ảnh RGB từ `/oakd/rgb/preview/image_raw`. Node `object_localization_node` subscribe `/oakd/rgb/preview/depth` và `/oakd/rgb/preview/camera_info` để tính vị trí mục tiêu 3D.

Bảng 4.1 Các topic cảm biến chính

Topic	Ý nghĩa
<code>/scan</code>	Dữ liệu LiDAR 2D dùng cho SLAM, localization và costmap.
<code>/odom</code>	Ước lượng chuyển động tương đối của robot.
<code>/oakd/rgb/preview/image_raw</code>	Ảnh RGB đầu vào cho YOLOv8.
<code>/oakd/rgb/preview/depth</code>	Ảnh chiều sâu để tính khoảng cách tới mục tiêu.
<code>/oakd/rgb/preview/camera_info</code>	Mã nội tại camera dùng cho chuyển đổi 2D-3D.

4.4 Nhận diện đối tượng bằng YOLOv8n

Node `tb4_vision_oak` tải mô hình YOLOv8n từ thư mục `models/yolov8n.pt`. Ngưỡng confidence mặc định là 0.5. Với mỗi ảnh RGB, node chạy inference, trích xuất bbox, class id và confidence, sau đó publish kết quả dưới dạng `vision_msgs/Detection2DArray` trên topic `/vision/detected_objects`.

Việc sử dụng YOLOv8n có ưu điểm là tốc độ xử lý nhanh và đủ chính xác cho bài toán phát hiện người trong mô phỏng. Trong code, lớp person có `class_id = 0` theo COCO dataset được ưu tiên khi có nhiều detection. Nếu không có người, detection có confidence cao nhất được chọn làm ứng viên.

4.5 Định vị mục tiêu từ RGB-D

Sau khi có bbox từ YOLOv8, node `object_localization_node` lấy tâm bbox (u, v) , đọc depth tại vùng lân cận tâm và dùng median để giảm ảnh hưởng pixel lỗi. Nếu depth hợp lệ, vị trí 3D trong hệ camera được tính theo mô hình pinhole:

$$X_c = \frac{(u - c_x)Z}{f_x}, \quad Y_c = \frac{(v - c_y)Z}{f_y}, \quad Z_c = Z$$

Sau đó, hệ thống ánh xạ trực camera sang quy ước ROS:

$$x_{ros} = Z_c, \quad y_{ros} = -X_c, \quad z_{ros} = -Y_c$$

Kết quả được publish qua `/target_object_pose_map` dưới dạng `PoseStamped` và `/target_object_marker` dưới dạng `Marker` để hiển thị trên RViz. Mặc dù tên topic có hậu tố `map`, frame ban đầu vẫn là `oakd_link`; Mission Manager sau đó dùng TF để chuyển quan hệ sang frame `map`.

4.6 TF và hợp nhất thông tin cảm biến

TF là cơ chế quan trọng giúp hợp nhất thông tin từ nhiều cảm biến. Khi Mission Manager nhận pose mục tiêu trong frame camera, nó dùng `tf2_ros.Buffer` để lookup transform từ `oakd_link` sang `map`. Nhờ đó, vị trí tương đối của mục tiêu có thể được chuyển thành vị trí toàn cục phục vụ lập kế hoạch đường đi.

Nếu TF không đầy đủ hoặc không đồng bộ thời gian, hệ thống sẽ không thể biến đổi mục tiêu sang bản đồ, dẫn tới Mission Manager không gửi được goal tiếp cận. Đây là lý do RViz cần hiển thị đồng thời TF, image, scan, map và marker trong quá trình debug.

4.7 Đánh giá kỹ thuật hệ cảm biến

Hệ cảm biến trong đề tài có tính bổ sung lẫn nhau. LiDAR ổn định cho định vị hình học và tránh vật cản, nhưng không cung cấp thông tin ngữ nghĩa. Camera RGB cung cấp thông tin ngữ nghĩa qua YOLO, nhưng thiếu khoảng cách nếu không có depth. Odometry có đáp ứng nhanh, nhưng tích lũy sai số theo thời gian. TF là lớp liên kết hình học giúp toàn bộ dữ liệu có thể sử dụng chung trong một hệ quy chiếu.

4.8 Kết luận chương

Chương này đã phân tích chi tiết hệ thống cảm biến của TurtleBot4 trong đề tài, bao gồm LiDAR, camera RGB-D, odometry, TF, YOLOv8 và định vị mục tiêu 3D. Đây là lớp thông tin đầu vào quyết định chất lượng của SLAM, Nav2 và Mission Manager.

5. CƠ CẤU CHẤP HÀNH VÀ ĐIỀU KHIỂN CHUYỂN ĐỘNG

5.1 Cấu trúc chấp hành của TurtleBot4

TurtleBot4 thuộc nhóm robot di động mặt đất sử dụng truyền động vi sai. Hai bánh chủ động được điều khiển độc lập, kết hợp với bánh tự do để giữ cân bằng cơ học. Cấu

trúc này đơn giản nhưng hiệu quả cho robot trong nhà, đặc biệt trong các nhiệm vụ tuần tra, điều hướng hành lang và tiếp cận mục tiêu.

Cơ cấu chấp hành của TurtleBot4 nhận lệnh vận tốc ở mức cao dưới dạng `geometry_msgs/Twist` thường là topic `/cmd_vel`. Lệnh gồm vận tốc tuyến tính theo trục x và vận tốc góc quanh trục z . Bộ điều khiển cấp thấp chuyển đổi lệnh này thành vận tốc từng bánh.

5.2 Mô hình động học robot vi sai

Với robot vi sai, giả sử robot chuyển động trong mặt phẳng, trạng thái robot được mô tả bởi (x, y, θ) . Phương trình động học là:

$$\dot{x} = v \cos \theta, \quad \dot{y} = v \sin \theta, \quad \dot{\theta} = \omega$$

Trong đó v là vận tốc tuyến tính, ω là vận tốc góc. Quan hệ giữa vận tốc robot và vận tốc bánh xe là:

$$v = \frac{r}{2}(\omega_r + \omega_l), \quad \omega = \frac{r}{L}(\omega_r - \omega_l)$$

Suy ra:

$$\omega_r = \frac{v + \frac{L}{2}\omega}{r}, \quad \omega_l = \frac{v - \frac{L}{2}\omega}{r}$$

Mô hình này cho thấy mỗi quyết định điều hướng cuối cùng đều được thực thi thông qua hai đại lượng vật lý là vận tốc bánh trái và bánh phải.

5.3 Nav2 như lớp điều khiển chuyển động cấp cao

Nav2 đóng vai trò bộ điều khiển chuyển động cấp cao. Khi nhận goal từ Patrol Node hoặc Mission Manager, Nav2 thực hiện các bước: kiểm tra bản đồ, lập đường đi toàn cục, tạo lệnh điều khiển cục bộ, tránh vật cản và gửi lệnh vận tốc cho robot. Trong hệ thống này, các node điều phối không điều khiển trực tiếp động cơ mà gửi mục tiêu vị trí cho Nav2 qua action `NavigateToPose`.

Cách thiết kế này có ưu điểm là tách biệt rõ nhiệm vụ và điều khiển. Mission Manager quyết định robot cần làm gì, còn Nav2 quyết định robot di chuyển như thế nào để đạt mục tiêu đó một cách an toàn.

5.4 Patrol Node

Patrol Node tạo hành vi tuần tra bằng cách gửi tuần tự các waypoint trong frame map. Danh sách waypoint hiện tại gồm bốn điểm tạo thành một chu trình đơn giản:

$$(1.0, 0.0), \quad (1.0, 1.0), \quad (0.0, 1.0), \quad (0.0, 0.0)$$

Node sử dụng action client `NavigateToPose`. Khi Nav2 hoàn thành một waypoint, node tăng chỉ số và gửi waypoint tiếp theo. Khi nhận tín hiệu `/mission_manager/patrol_enabled = false`, Patrol Node hủy goal hiện tại để nhường quyền điều khiển cho Mission Manager.

5.5 Mission Manager và điều khiển hành vi

Mission Manager là lớp điều phối nhiệm vụ. Khi robot đang tuần tra và phát hiện mục tiêu ở khoảng cách lớn hơn `safe_distance`, node tạm dừng tuần tra, biến đổi vị trí mục tiêu sang frame map, tính điểm tiếp cận an toàn và gửi goal mới cho Nav2.

Điểm tiếp cận không trùng với vị trí mục tiêu mà nằm cách mục tiêu một khoảng an toàn. Nếu vector từ robot tới mục tiêu là (d_x, d_y) , khoảng cách phẳng là:

$$d = \sqrt{d_x^2 + d_y^2}$$

Với khoảng cách an toàn d_s , điểm goal được tính theo tỷ lệ:

$$\alpha = \frac{d - d_s}{d}$$

$$x_g = x_r + \alpha d_x, \quad y_g = y_r + \alpha d_y$$

Cách tính này giúp robot tiếp cận mục tiêu mà không va chạm hoặc đứng quá sát đối tượng.

5.6 Quan hệ giữa cảm biến và chấp hành

Điểm cốt lõi của hệ thống là liên hệ giữa cảm biến và cơ cấu chấp hành. Cảm biến phát hiện mục tiêu; thuật toán định vị chuyển mục tiêu thành tọa độ; Mission Manager biến tọa độ thành goal; Nav2 biến goal thành lệnh vận tốc; cơ cấu vi sai biến lệnh vận tốc thành chuyển động bánh xe. Chuỗi này thể hiện rõ bản chất của một hệ robot tự hành:

Detection → 3D Localization → Goal → Velocity Command → Wheel Motion

5.7 Kết luận chương

Chương này đã trình bày cơ cấu chấp hành của TurtleBot4 và cách hệ thống điều khiển chuyển động thông qua Nav2, Patrol Node và Mission Manager. Phân tích cho thấy cơ cấu chấp hành không hoạt động độc lập mà luôn phụ thuộc vào dữ liệu cảm biến và quyết định điều hướng cấp cao.

6. TRIỂN KHAI NHIỆM VỤ TỰ HÀNH

6.1 Mục tiêu nhiệm vụ

Nhiệm vụ tự hành trong đề tài được thiết kế theo kịch bản: robot tuần tra trong môi trường mô phỏng, quan sát bằng camera RGB-D, phát hiện mục tiêu bằng YOLOv8, định vị mục tiêu trong không gian 3D, tạm dừng tuần tra, tiếp cận mục tiêu ở khoảng cách an toàn và sau đó quay lại chế độ tuần tra.

Kịch bản này phù hợp với các ứng dụng như robot giám sát kho hàng, robot tuần tra văn phòng, robot hỗ trợ tìm kiếm hoặc robot dịch vụ có khả năng phản ứng với đối tượng trong môi trường.

6.2 Quy trình khởi động hệ thống

Hệ thống được khởi động theo hai bước chính. Trước hết, mô phỏng TurtleBot4 được launch trong Gazebo/Ignition. Sau đó, launch tổng hợp `sim_demo.launch.py` được sử dụng để chạy Nav2, localization, vision, object localization, patrol và mission manager.

```
1 cd ~/tb4_project_ab
2 export ROS_DOMAIN_ID=7
3 export RMW_FASTRTPS_USE_SHM=0
4 source /opt/ros/humble/setup.bash
5 source install/setup.bash
6
7 ros2 launch turtlebot4_ignition_bringup
   turtlebot4_ignition.launch.py \
8   model:=lite world:=warehouse x:=2.0 y:=0.0 z:=0.01
   yaw:=0.0
```

Listing 1 Lệnh khởi động mô phỏng TurtleBot4

```
1 cd ~/tb4_project_ab
2 export ROS_DOMAIN_ID=7
3 export RMW_FASTRTPS_USE_SHM=0
4 source /opt/ros/humble/setup.bash
5 source install/setup.bash
6
7 ros2 launch tb4_bringup sim_demo.launch.py use_rviz:=true
```

Listing 2 Lệnh chạy full mission

6.3 SLAM và bản đồ môi trường

Trong giai đoạn mapping, SLAM Toolbox sử dụng dữ liệu /scan, odometry và TF để xây dựng occupancy grid map. Mỗi ô trong bản đồ biểu diễn xác suất bị chiếm bởi vật cản. Bản đồ sau khi tạo được lưu lại và dùng cho giai đoạn localization/navigation.

Các tham số quan trọng trong cấu hình SLAM gồm:

- resolution: 0.05: độ phân giải bản đồ 5 cm mỗi ô.
- min_laser_range: 0.2: loại bỏ đo quá gần.
- max_laser_range: 12.0: giới hạn tầm đo LiDAR.
- do_loop_closing: true: cho phép phát hiện đóng vòng để giảm sai số tích lũy.
- use_scan_matching: true: dùng scan matching để cải thiện ước lượng pose.

6.4 Navigation và tuần tra

Sau khi có bản đồ, Nav2 được dùng để điều hướng. Patrol Node gửi các goal tuần tra tuần tự đến action server navigate_to_pose. Trạng thái goal được kiểm tra thông qua GoalStatus. Nếu goal thành công, robot chuyển sang waypoint tiếp theo; nếu bị hủy bởi Mission Manager, node dừng tuần tra tạm thời.

Việc sử dụng action thay vì topic đơn giản giúp quá trình điều hướng có phản hồi rõ ràng: goal được chấp nhận hay từ chối, đang chạy hay đã hoàn thành. Đây là thiết kế phù hợp cho hệ robot tự hành vì navigation là quá trình kéo dài theo thời gian, không phải một lệnh tức thời.

6.5 Nhận diện và định vị mục tiêu

Node YOLOv8 xử lý ảnh RGB theo thời gian thực và publish detection. Node localization chọn detection phù hợp, lấy depth trung vị tại vùng tâm bbox để giảm nhiễu, sau đó tính pose tương đối. Marker được hiển thị trên RViz giúp kiểm tra trực quan vị trí mục tiêu.

Quy trình này minh họa rõ sự kết hợp giữa cảm biến thị giác và cảm biến chiều sâu. Nếu chỉ dùng ảnh RGB, hệ thống chỉ biết mục tiêu nằm ở đâu trên ảnh. Nếu chỉ dùng depth, hệ thống không biết điểm nào là mục tiêu. Khi kết hợp RGB, YOLO và depth, robot có thể chuyển thông tin ngữ nghĩa thành tọa độ không gian.

6.6 Mission Manager

Mission Manager có trạng thái chính là patrolling và approaching_target. Khi đang tuần tra, nếu nhận được vị trí mục tiêu hợp lệ và mục tiêu ở xa hơn khoảng cách an toàn, node gửi tín hiệu tạm dừng tuần tra qua /mission_manager/patrol_enabled. Sau đó, nó tính goal tiếp cận an toàn và gửi goal này đến Nav2.

Sau khi goal tiếp cận kết thúc, Mission Manager đặt thời gian cooldown để tránh phản ứng lặp lại liên tục với cùng mục tiêu, rồi bật lại tuần tra. Thiết kế này tuy đơn giản nhưng đủ thể hiện mô hình điều phối hành vi cơ bản của robot tự hành.

6.7 Các topic kiểm tra quan trọng

Trong quá trình vận hành, các topic sau cần được kiểm tra để đánh giá hệ thống:

```
1 ros2 topic echo /scan --once
2 ros2 topic echo /odom --once
3 ros2 topic echo /clock --once
4 ros2 topic echo /vision/detected_objects --once
5 ros2 topic echo /target_object_pose_map --once
6 ros2 topic echo /target_object_marker --once
```

Listing 3 Các lệnh kiểm tra topic chính

Nếu các topic trên có dữ liệu ổn định, có thể kết luận rằng pipeline cảm biến–nhận diện–định vị–điều hướng đang hoạt động đúng về mặt truyền thông ROS2.

6.8 Kết luận chương

Chương này đã trình bày cách triển khai nhiệm vụ tự hành hoàn chỉnh trên TurtleBot4, từ khởi động mô phỏng, xây dựng bản đồ, điều hướng, tuần tra, nhận diện mục tiêu đến tiếp cận mục tiêu. Đây là chương kết nối các phân tích lý thuyết về cảm biến và cơ cấu chấp hành với hệ thống thực thi cụ thể.

7. KẾT QUẢ MÔ PHỎNG VÀ ĐÁNH GIÁ

7.1 Kết quả đạt được

Hệ thống đã được xây dựng theo kiến trúc ROS2 module hóa và có thể khởi chạy luồng nhiệm vụ tự hành trong môi trường mô phỏng. Các kết quả chính gồm:

- TurtleBot4 Lite spawn thành công trong môi trường warehouse của Gazebo/Ignition.
- LiDAR, camera RGB-D, odometry và TF hoạt động, cung cấp dữ liệu cho các node xử lý.
- SLAM Toolbox có thể sử dụng dữ liệu LiDAR để xây dựng bản đồ occupancy grid.
- Nav2 có thể nhận goal và điều hướng robot theo waypoint.
- YOLOv8n được tích hợp để phát hiện mục tiêu từ ảnh RGB.
- Object Localization tính được vị trí mục tiêu tương đối từ detection và depth.
- Marker mục tiêu có thể hiển thị trên RViz.
- Mission Manager điều phối được quá trình tuần tra, phát hiện và tiếp cận mục tiêu.

7.2 Đánh giá hệ cảm biến

Hệ cảm biến đáp ứng được yêu cầu cơ bản của bài toán. LiDAR cung cấp dữ liệu đủ cho SLAM và costmap. Camera RGB-D hỗ trợ nhận diện và định vị mục tiêu. Odometry và TF cung cấp liên kết trạng thái cần thiết cho navigation. Tuy nhiên, hệ thống phụ thuộc mạnh vào chất lượng TF và đồng bộ thời gian. Nếu transform giữa camera và map không sẵn sàng, Mission Manager không thể chuyển mục tiêu sang tọa độ toàn cục.

Một điểm cải thiện đáng chú ý là xử lý depth. Việc dùng median trong cửa sổ quanh tâm bbox giúp tăng độ ổn định so với lấy một pixel đơn lẻ. Đây là lựa chọn hợp lý vì ảnh depth thường có điểm mất dữ liệu hoặc nhiễu cục bộ tại biên vật thể.

7.3 Đánh giá cơ cấu chấp hành và điều khiển

Về lớp chấp hành, robot vi sai phù hợp với môi trường trong nhà và nhiệm vụ tuần tra. Nav2 giúp giảm đáng kể độ phức tạp khi điều khiển vì hệ thống không cần tự viết bộ điều khiển local planner từ đầu. Patrol Node và Mission Manager chỉ cần gửi goal, còn Nav2 xử lý lập đường, tránh vật cản và tạo lệnh vận tốc.

Tuy nhiên, hệ thống hiện chưa tối ưu sâu các tham số planner, controller và costmap. Với môi trường phức tạp hoặc nhiều vật cản động, cần tinh chỉnh thêm để tăng độ mượt, giảm dao động và tránh kẹt cục bộ.

7.4 Đánh giá kiến trúc phần mềm

Kiến trúc module hóa là điểm mạnh của đề tài. Mỗi package có trách nhiệm rõ ràng, giúp dễ debug và mở rộng. Ví dụ, có thể thay YOLOv8n bằng một mô hình custom mà không cần thay đổi Nav2; hoặc có thể thay Patrol Node bằng behavior tree mà không ảnh hưởng tới object localization.

Cơ chế publish/subscribe và action của ROS2 phù hợp với hệ thống phân tán. Topic dùng cho dữ liệu liên tục như ảnh, scan, pose; action dùng cho nhiệm vụ kéo dài như navigation goal; parameter dùng cho cấu hình như confidence threshold và safe distance.

7.5 Hạn chế

Bên cạnh các kết quả đạt được, hệ thống vẫn còn một số hạn chế:

- Chưa triển khai và kiểm thử trên TurtleBot4 thật.
- Waypoint tuần tra còn hardcoded trong source code.
- Mission Manager mới ở dạng logic trạng thái đơn giản, chưa phải behavior tree hoặc finite state machine đầy đủ.
- Chưa tối ưu toàn diện tham số Nav2, costmap, planner và controller.
- Chưa có cơ chế tracking nhiều mục tiêu hoặc mục tiêu chuyển động.
- Pose mục tiêu phụ thuộc vào depth tại bbox, dễ bị ảnh hưởng khi vật thể bị che khuất hoặc depth không ổn định.
- Robot chưa tối ưu hướng quay mặt về phía mục tiêu khi tiếp cận.

7.6 Hướng phát triển

Các hướng phát triển tiếp theo gồm:

- Triển khai trên robot TurtleBot4 thật và đánh giá sai số cảm biến thực tế.
- Đưa waypoint tuần tra ra file YAML để cấu hình linh hoạt.
- Xây dựng Mission Manager theo finite state machine hoặc behavior tree.
- Tối ưu Nav2 costmap, planner và controller cho từng môi trường.
- Thêm multi-object tracking và lọc vị trí mục tiêu theo thời gian.
- Huấn luyện YOLO custom cho các đối tượng chuyên biệt.
- Tích hợp semantic map để robot hiểu thêm ý nghĩa của môi trường.
- Bổ sung logging và tiêu chí định lượng như sai số định vị, thời gian tiếp cận, tỷ lệ phát hiện đúng và độ ổn định navigation.

7.7 Kết luận chương

Kết quả mô phỏng cho thấy hệ thống đã đáp ứng mục tiêu đề tài ở mức tích hợp chức năng. Robot có thể mô phỏng, thu nhận dữ liệu cảm biến, lập bản đồ, điều hướng, nhận diện mục tiêu và thực hiện hành vi tiếp cận. Các hạn chế hiện tại chủ yếu nằm ở mức tối ưu, đánh giá định lượng và triển khai thực nghiệm trên phần cứng thật.

8. KẾT LUẬN

8.1 Kết luận chung

Báo cáo đã trình bày quá trình nghiên cứu và xây dựng hệ thống robot tự hành TurtleBot4 trong môi trường mô phỏng sử dụng ROS2, SLAM, Nav2 và YOLOv8. Dưới góc nhìn của học phần *Cảm biến và Cơ cấu chấp hành*, đề tài thể hiện rõ mối liên hệ giữa hệ thống đo lường, xử lý thông tin, lập kế hoạch và thực thi chuyển động.

Hệ thống đã tích hợp được các cảm biến quan trọng gồm LiDAR, camera RGB-D, odometry và TF. LiDAR đóng vai trò chính trong SLAM và navigation; camera RGB-D hỗ trợ nhận diện và định vị mục tiêu; odometry cung cấp thông tin chuyển động liên tục; TF đảm bảo quan hệ hình học giữa các frame. Các nguồn dữ liệu này được xử lý bởi ROS2 để tạo thành thông tin có ý nghĩa phục vụ điều khiển robot.

Về cơ cấu chấp hành, TurtleBot4 sử dụng truyền động vi sai, nhận lệnh vận tốc từ Nav2 và thực hiện chuyển động trong môi trường. Nav2 đóng vai trò lớp điều khiển chuyển động cấp cao, chuyển goal từ Patrol Node hoặc Mission Manager thành lệnh vận tốc phù hợp. Mission Manager cho phép robot chuyển đổi hành vi từ tuần tra sang tiếp cận mục tiêu, qua đó minh họa cách cảm biến ảnh hưởng trực tiếp tới quyết định chấp hành.

8.2 Các nội dung đã hoàn thành

Trong quá trình thực hiện, báo cáo đã hoàn thành các nội dung sau:

- Phân tích bối cảnh, mục tiêu và phạm vi của bài toán robot tự hành TurtleBot4.
- Trình bày cơ sở lý thuyết về cảm biến, cơ cấu chấp hành, SLAM, Nav2, YOLOv8 và robot vi sai.
- Thiết kế kiến trúc hệ thống ROS2 theo hướng module hóa.
- Phân tích pipeline cảm biến gồm LiDAR, RGB-D camera, odometry và TF.
- Phân tích cách nhận diện mục tiêu bằng YOLOv8n và định vị mục tiêu bằng ảnh depth.
- Phân tích cơ cấu chấp hành vi sai và luồng điều khiển chuyển động thông qua Nav2.
- Triển khai logic nhiệm vụ gồm tuần tra, phát hiện mục tiêu, tiếp cận an toàn và quay lại tuần tra.
- Đánh giá kết quả mô phỏng, hạn chế và hướng phát triển.

8.3 Ý nghĩa đối với học phần

Đề tài giúp củng cố kiến thức của học phần theo hướng ứng dụng thực tế. Thay vì khảo sát cảm biến và cơ cấu chấp hành một cách riêng lẻ, báo cáo đặt chúng trong một hệ robot hoàn chỉnh. Qua đó, có thể thấy cảm biến quyết định robot biết gì về môi trường, còn cơ cấu chấp hành quyết định robot có thể làm gì trong môi trường đó. Giữa hai lớp này là các thuật toán xử lý, lập kế hoạch và điều khiển.

Cách tiếp cận này phù hợp với xu hướng robotics hiện đại, nơi kỹ sư không chỉ cần hiểu từng linh kiện mà còn phải hiểu cách tích hợp cảm biến, phần mềm và truyền động thành một hệ thống đáng tin cậy.

8.4 Hạn chế và định hướng tiếp theo

Do giới hạn thời gian và điều kiện phần cứng, đề tài mới dừng ở mức mô phỏng. Các yếu tố thực tế như nhiễu cảm biến thật, sai số lắp đặt, trượt bánh, độ trễ truyền thông, giới hạn moment động cơ và điều kiện ánh sáng chưa được đánh giá đầy đủ. Trong tương lai, cần triển khai trên TurtleBot4 thật, bổ sung đánh giá định lượng và tối ưu tham số điều hướng.

Ngoài ra, Mission Manager có thể được nâng cấp thành behavior tree, hỗ trợ nhiều trạng thái phức tạp hơn như tìm kiếm mục tiêu, theo dõi mục tiêu động, tránh vùng cấm, quay về trạm sạc và phối hợp nhiều robot. Đây là các hướng mở rộng có giá trị thực tiễn cao.

8.5 Lời kết

Thông qua đề tài này, em đã hiểu sâu hơn vai trò của cảm biến và cơ cấu chấp hành trong hệ robot tự hành, đồng thời có cơ hội thực hành với ROS2, Gazebo/Ignition, SLAM, Nav2 và YOLOv8. Những kiến thức và kinh nghiệm này là nền tảng quan trọng để tiếp tục nghiên cứu các hệ robot thông minh trong tương lai.

PHỤ LỤC A: CẤU TRÚC SOURCE CODE

```
1 Robots_Sensor_Actuators/
2 |-- tb4_bringup/
3 |   |-- launch/sim_demo.launch.py
4 |   |-- launch/slam_demo.launch.py
5 |   |-- config/slam_toolbox_tb4.yaml
6 |   |-- rviz/tb4_debug.rviz
7 |   '-- tb4_bringup/initial_pose_publisher.py
8 |-- tb4_vision_oak/
9 |   |-- launch/oak_detection.launch.py
10 |   |-- config/camera_params.yaml
11 |   |-- models/yolov8n.pt
12 |   '-- tb4_vision_oak/detection_node.py
13 |-- tb4_object_localization/
14 |   |-- launch/localization.launch.py
15 |   '-- scripts/localization_node.py
16 |-- tb4_nav_patrol/
17 |   |-- launch/nav_patrol.launch.py
18 |   '-- tb4_nav_patrol/patrol_node.py
19 '-- tb4_mission_manager/
20     |-- launch/mission_manager.launch.py
21     '-- tb4_mission_manager/mission_manager_node.py
```

PHỤ LỤC B: CÁC LỆNH CHẠY CHÍNH

Build workspace

```
1 cd ~/tb4_project_ab
2 source /opt/ros/humble/setup.bash
3 colcon build --symlink-install
4 source install/setup.bash
```

Launch simulator

```
1 cd ~/tb4_project_ab
2 export ROS_DOMAIN_ID=7
3 export RMW_FASTRTPS_USE_SHM=0
4 source /opt/ros/humble/setup.bash
5 source install/setup.bash
6
7 ros2 launch turtlebot4_ignition_bringup
  turtlebot4_ignition.launch.py \
8   model:=lite world:=warehouse x:=2.0 y:=0.0 z:=0.01
  yaw:=0.0
```

Chạy full mission

```
1 cd ~/tb4_project_ab
2 export ROS_DOMAIN_ID=7
3 export RMW_FASTRTPS_USE_SHM=0
4 source /opt/ros/humble/setup.bash
5 source install/setup.bash
6
7 ros2 launch tb4_bringup sim_demo.launch.py use_rviz:=true
```

Kiểm tra topic

```
1 ros2 topic echo /scan --once
2 ros2 topic echo /odom --once
3 ros2 topic echo /clock --once
4 ros2 topic echo /vision/detected_objects --once
```

```
5 ros2 topic echo /target_object_pose_map --once
6 ros2 topic echo /target_object_marker --once
```

PHỤ LỤC C: MỘT SỐ LỖI VÀ CÁCH KHẮC PHỤC

Lỗi ROS daemon hoặc session cũ

- Dừng ROS daemon bằng `ros2 daemon stop`.
- Kill các process ROS2/Gazebo cũ nếu còn chạy.
- Xóa cache shared memory nếu cần.
- Source lại môi trường ROS2 và workspace.

Lỗi /scan bất thường

- Kiểm tra topic /scan có dữ liệu hợp lệ hay không.
- Kiểm tra frame và vị trí lắp đặt `rplidar_link`.
- Kiểm tra collision giữa LiDAR và robot model.
- Quan sát LaserScan trong RViz để xác định lỗi hình học.

Lỗi không thấy marker mục tiêu

- Kiểm tra /vision/detected_objects có detection hay không.
- Kiểm tra ảnh depth và camera info có publish hay không.
- Kiểm tra /target_object_pose_map có dữ liệu hay không.
- Trong RViz thêm display Marker với topic /target_object_marker.

TÀI LIỆU THAM KHẢO

1. TS. Phạm Hoàng Anh, *Bài giảng học phần Cảm biến và Cơ cấu chấp hành*, Khoa Kỹ thuật Điện tử 1, Học viện Công nghệ Bưu chính Viễn thông.
2. Open Robotics, *ROS2 Documentation*, ROS2 Humble Hawksbill.
3. Open Navigation, *Nav2 Documentation*, Navigation2 project.
4. Slam Toolbox, *ROS2 SLAM Toolbox Documentation*.
5. TurtleBot4, *TurtleBot4 User Manual and Simulation Documentation*, Clearpath Robotics / Open Robotics.
6. Ultralytics, *YOLOv8 Documentation*.
7. R. Siegwart, I. R. Nourbakhsh, D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, MIT Press.
8. B. Siciliano, L. Sciavicco, L. Villani, G. Oriolo, *Robotics: Modelling, Planning and Control*, Springer.